Table of Contents

INTRODUCTION 2

Overall Workflow 2
Backend Overview 2

INSTALLATION OF NUGET PACKAGES 4

MODIFYING APPSETTINGS.JSON 5

MODIFYING PROGRAM.CS 7

Using statements 7
Adding DbContext 8
Adding the services for authorisation and authentication 9
Adding the authentication and authorisation middlewares 11

ADD MODEL 13

ADD DBCONTEXT 14

ADD DTOS 15

ADD AUTHCONTROLLER 16

Add the api/Controllers/AuthController.cs 16
Dependency Injection 17
Register Endpoint 18
Login Endpoint 19
Logout Endpoint 19
GenerateJwtToken Endpoint 20

MODIFYING ITEMCONTROLLER 21

TEST THE BACKEND API VIA SCALAR 22

Demo: JWT Authentication-backend

Author: Baifan Zhou
Detailed explanations are created with the help of ChatGPT/Gemini/Claude.

Introduction

This demo will guide you through setting up a secure backend API with JWT (JSON Web Token) authentication using ASP.NET Core.
JWT (JSON Web Token) is a popular authentication technique for frontend-backend communication. JWT is a modern, stateless authentication method where the server generates a signed token for an authenticated user, and the client stores and sends this token with every request to access protected resources.

Overall Workflow

User Registration

- User submits registration form through frontend
- Backend validates and stores user credentials in database
- No token is generated during registration

User Login

- User submits login credentials through frontend
- Backend verifies credentials against database. If valid, backend generates a signed JWT token containing user information
- Frontend receives and stores the JWT token

Accessing Protected Routes

- User attempts to access protected resources (usually Create/Update/Delete operations)
- Frontend automatically includes JWT token in the Authorization header of API requests
- Backend validates the JWT token's signature and expiration
    - *Valid token*: Request proceeds to the protected endpoint
    - *Invalid/expired token*: Request rejected with 401 Unauthorized

User Logout

- Frontend removes the JWT token from **localStorage**
- Backend
    - No backend communication required for JWT (stateless nature of JWT)
    - Note: SignOutAsync() is used for cookie-based authentication, not JWT. Key Point: Unlike session-based authentication, JWT is stateless. The backend *doesn't store active sessions*, making it ideal for scalable web APIs.

Backend Overview

To realise the workflow, we need a couple of files. For the backend authentication, we need **AuthDbContext**, **AuthController**, **AuthUser data model**, **data transfer objects** (DTOs) for user registration and login, and modification of **ItemController.cs** to protect the create/update.delete, and finally **Program.cs** and **appsettings.json**.

Below is the list with concise explanation for the Backend (.NET API)

- **api/Program.cs**: Configures and enables the JWT authentication and authorisation middleware. This is where the API is told how to validate the tokens it receives.
- **api/appsettings.json**: Stores the secret Key, Issuer, and Audience for the JWT. Keeping this configuration here makes it secure and easy to change.
- **api/Controllers/AuthController.cs**: Handles the /api/Auth/login and /api/Auth/register endpoints. Its primary job is to validate user credentials and generate a signed JWT for the client upon successful login.
- **api/Controllers/ItemController.cs**: adding [Authorize] to create/update/delete to protect them from unauthorised access.
- **api/Models/AuthUser.cs**: Defines the user entity that is managed by ASP.NET Identity, including properties like UserName, Email, etc.
- **api/DTOs/LoginDto.cs & RegisterDto.cs**: These are Data Transfer Objects that define the exact structure of the JSON data the frontend sends when a user tries to log in or register.
- **api/DAL/AuthDbContext.cs**: The Entity Framework database context that connects ASP.NET Identity to your user database, allowing it to create, read, and validate users.

Installation of NuGet packages

We start with installing 3 important packages. When installing under the folder **api**, using the below command with specified version number to ensure compatibility across different machines. You can use nuget.org to check the latest version numbers.

```
cd api
dotnet add package <package full name> --version=<version number>

dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer --
version=10.0.11
dotnet add package Microsoft.IdentityModel.Tokens --version=8.22.0
dotnet add package System.IdentityModel.Tokens.Jwt --version=8.22.0
```

Your api.csproj should include three more packages as below:

```
<PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer"
Version="10.0.11" />
<PackageReference Include="Microsoft.IdentityModel.Tokens" Version="8.22.0"
/>
<PackageReference Include="System.IdentityModel.Tokens.Jwt"
Version="8.22.0" />
```

- **Microsoft.AspNetCore.Authentication.JwtBearer**: This is the core **middleware package** that integrates JWT authentication into your ASP.NET Core application's request pipeline. It provides the **AddJwtBearer** and **UseAuthentication** methods that configure the framework to look for, read, and validate JWTs from incoming requests.
- **System.IdentityModel.Tokens.Jwt**: This is the **token handling package**. It contains the classes for programmatically creating, reading, and writing JSON Web Tokens. The key class here is **JwtSecurityTokenHandler**, which is responsible for the actual manipulation of the JWT strings.
- **Microsoft.IdentityModel.Tokens**: This is the **cryptographic package**. It provides the classes for the underlying security primitives, such as the cryptographic keys (SymmetricSecurityKey) and the signing credentials (SigningCredentials) that are used to secure the token and verify its integrity.

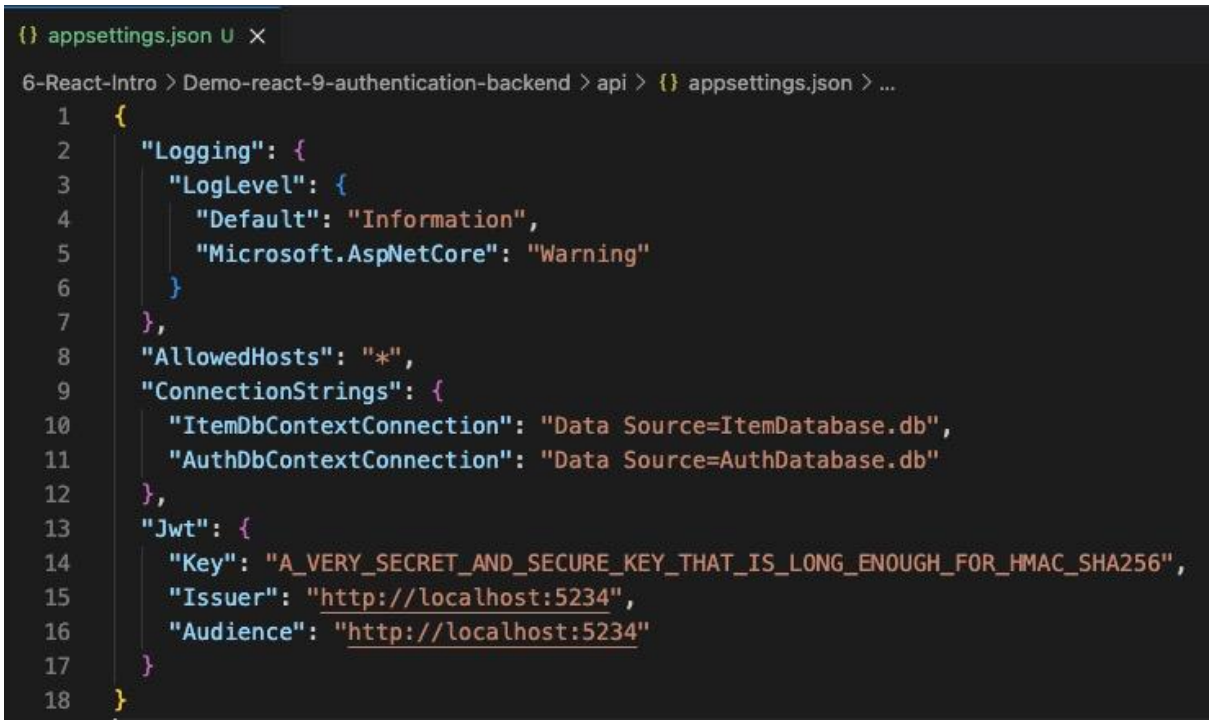
Modifying appsettings.json

Now we modify this file to specify which database we use and the JWT configurations.

Before:



After:



Note: in real development, you can use `dotnet user-secrets` to manage your key (search by yourself to learn it), which stores config values *outside* your project folder (in a per-project JSON file under your OS profile. Here in the demo, to ensure this project can run across different machines without further modifications, we simply use this dummy key.

AuthDbContextConnection: This is a connection string that tells the application how to connect to the database. Data Source=AuthDatabase.db specifies that you're using an **SQLite database file** named AuthDatabase.db to store user-related data.

Jwt: This section contains the configuration settings for your **JSON Web Token (JWT)**.

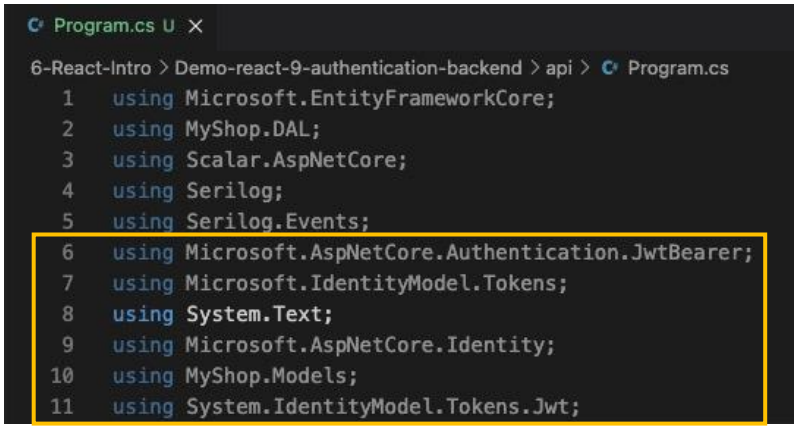
- **Key**: The secret key used to **sign the token**. This key ensures the token's authenticity and integrity, so it cannot be tampered with. It must be kept secret and should be long enough for the HMAC-SHA256 algorithm. Replace it to a more secure key in production.

- **Issuer**: The entity that issues the token. Here, it's your own API's URL.
- **Audience**: The intended recipient of the token. In this case, it's also your own API, meaning the token is meant for use within your application.

Modifying Program.cs

Using statements

We start with adding the **using** statements.
After:



We added three groups of using statements to support the new authentication system.

1. **Microsoft.AspNetCore.Authentication.JwtBearer**, **Microsoft.IdentityModel.Tokens**, **System.Text**: These are required for the JWT (JSON Web Token) authentication middleware. They provide the classes to configure the JWT Bearer authentication scheme, which validates and processes the tokens sent by the client.
2. **Microsoft.AspNetCore.Identity**: This is the core namespace for ASP.NET Core Identity. It provides access to the IdentityUser, IdentityRole, UserManager, and SignInManager classes, which are essential for managing users and their credentials.
3. **MyShop.Models**: This statement gives you access to your custom AuthUser class, which extends IdentityUser, and any other user-related models you've defined. It allows you to correctly configure the Identity system to use your custom user model.

In addition, **System.IdentityModel.Tokens.Jwt** package is optional for debugging purpose. It contains the JwtSecurityTokenHandler class. This class is the engine that handles the token itself. The middleware uses it to perform key tasks like:

- Reading: Parsing the JWT string from the request.
- Creating: Building a new JwtSecurityToken object from the token's claims and credentials.
- Validating: Verifying the token's signature to ensure it hasn't been tampered with.
- In short, while other packages handle the configuration and pipeline, this specific package provides the core functionality for interacting with the JWT data format.

Adding DbContext

we add a new AuthDbContext after the ItemDbContext for accessing our authentication database, which stores the user credentials. Currently it has red underline because we have not defined the AuthDbContext. We define it soon!

Before:

```
builder.Services.AddDbContext<ItemDbContext>(options =>
{
    options.UseSqlite(builder.Configuration.GetConnectionString("ItemDbContextConnection"));
});
```

After:

```
builder.Services.AddDbContext<ItemDbContext>(options =>
{
    options.UseSqlite(builder.Configuration.GetConnectionString("ItemDbContextConnection"));
});

builder.Services.AddDbContext<AuthDbContext>(options =>
{
    options.UseSqlite(builder.Configuration.GetConnectionString("AuthDbContextConnection"));
});

builder.Services.AddIdentity<AuthUser, IdentityRole>()
    .AddEntityFrameworkStores<AuthDbContext>()
    .AddDefaultTokenProviders();
```

This code sets up the core services for user management and authentication using ASP.NET Core Identity.

- **AddDbContext<AuthDbContext>**: This line registers your AuthDbContext as a service, making it available throughout the application via dependency injection. It's configured to use SQLite with the connection string you defined in your appsettings.json, so it knows where to store your user data.
- **AddIdentity<AuthUser, IdentityRole>**: This is the main part of the setup. It adds and configures the ASP.NET Core Identity services.
    - **<AuthUser, IdentityRole>**: These generic types tell the Identity system to use your custom AuthUser model for user accounts and the default IdentityRole for user roles.
    - **.AddEntityFrameworkStores<AuthDbContext>()**: This line connects Identity to Entity Framework Core, instructing it to use your AuthDbContext to manage all user, role, and related data in the database.
    - **.AddDefaultTokenProviders()**: This adds the necessary services for generating tokens, which are used for things like password resets and email confirmations.

Adding the services for authorisation and authentication

```
builder.Services.AddAuthorization();
builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
    options.DefaultScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.SaveToken = true;
    options.RequireHttpsMetadata = false; // Set to true in production
    options.TokenValidationParameters = new TokenValidationParameters()
    {
        ValidateIssuer = true, // Validate the token issuer is correct
        ValidateAudience = true, // Validate the token reciepient is correct
        ValidateLifetime = true, // Validate the token has not expired
        ValidateIssuerSigningKey = true, // Validate the JWT signature
        ValidIssuer = builder.Configuration["Jwt:Issuer"], // Reading the issuer of the token
        ValidAudience = builder.Configuration["Jwt:Audience"], // Reading the audience for the token
        IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(
            builder.Configuration["Jwt:Key"] ?? throw new InvalidOperationException("JWT key not found in configuration.") //
            Reading the key from the configuration
        ))
    };
});
```

This code sets up and configures the services for **authentication** and **authorization** in your application.

Authentication and Authorization Services

The AddAuthentication and AddAuthorization services work together to secure your API.

AddAuthorization(): This line enables the authorization system, which is responsible for checking if an authenticated user has the necessary permissions to access a resource (e.g., via the [Authorize] attribute on a controller).

AddAuthentication(): This configures the authentication service to use **JWT Bearer** tokens as the primary method for authenticating users. It tells the application to look for and validate JWTs in the Authorization header of incoming requests.

AddJwtBearer(): This is where the JWT-specific rules are defined.

- options.SaveToken = true; saves the received token in memory.
- options.RequireHttpsMetadata = false; is a setting for development, allowing the token to be sent over HTTP. **This should be set to true in production.**
- options.TokenValidationParameters is the most important section, where you define the validation rules for the token. Each property, like **ValidateIssuer** and **ValidateAudience**, ensures the token is valid, comes from a trusted source, and hasn't been tampered with or expired. It uses the Jwt settings from your **appsettings.json** file to perform these checks.

```
    options.Events = new JwtBearerEvents
    {
        OnMessageReceived = context =>
        {
            var token = context.Request.Headers["Authorization"].FirstOrDefault()?.Split(" ").LastOrDefault();
            Console.WriteLine($"OnMessageReceived - Token: {token}");
            return Task.CompletedTask;
        },
        OnTokenValidated = context =>
        {
            Console.WriteLine("OnTokenValidated: SUCCESS");
            return Task.CompletedTask;
        },
        OnAuthenticationFailed = context =>
        {
            Console.WriteLine($"OnAuthenticationFailed: {context.Exception.Message}");
            Console.WriteLine($"Exception Type: {context.Exception.GetType().Name}");
            if (context.Exception.InnerException != null)
            {
                Console.WriteLine($"Inner Exception: {context.Exception.InnerException.Message}");
            }
            return Task.CompletedTask;
        },
        OnChallenge = context =>
        {
            Console.WriteLine($"OnChallenge: {context.Error} - {context.ErrorDescription}");
            return Task.CompletedTask;
        }
    };
```

The commented-out code is an event-based system that allows you to hook into different stages of the JWT authentication process for debugging purposes.

If you encounter bugs during the implementation of the JWT authentication, comment these code back and study the following code and console prints for debugging.

options.Events: This property allows you to register an instance of `JwtBearerEvents`, which contains a set of Task-returning methods that are invoked at specific points during authentication.

- **OnMessageReceived**: This event is triggered as soon as a request arrives, after the JWT has been extracted from the Authorization header. Your code logs the token here, which is useful for checking if the token is being sent correctly by the client.
- **OnTokenValidated**: This event fires if the JWT is successfully validated. This is a good place to log a success message, confirming that the token's signature, issuer, and expiration are all valid.
- **OnAuthenticationFailed**: This event is triggered when authentication fails for any reason (e.g., the token is expired, invalid, or missing). Your code logs the specific exception, which is crucial for debugging authentication problems on the server.
- **OnChallenge**: This event is called when an unauthenticated request attempts to access an authorized resource. It's used to send an appropriate response (like a 401 Unauthorized status code) back to the client.

In short, these events are a powerful debugging tool that gives you visibility into every step of the JWT validation pipeline.

Adding the authentication and authorisation middlewares

After adding:



We have added the core authentication and authorization middleware to the application's request pipeline. This is a critical step for securing your API.

- **app.UseAuthentication()**: This middleware is responsible for authenticating the user. It reads the JWT token from the incoming request, validates it using the rules you defined in Program.cs, and creates a User principal object. This object contains the user's claims and identity information, making it available to the rest of the application.
- **app.UseAuthorization()**: This middleware checks if the authenticated user is authorized to access a specific endpoint. It uses the User principal created by the UseAuthentication middleware and checks it against any [Authorize] attributes on your controllers or action methods.

Importance of Order

The order of this middleware is crucial and cannot be changed. The pipeline works sequentially, and each middleware relies on the one before it.

1. **MapStaticAssets()**: It must be placed before any middleware that needs to access static files, and it's typically one of the first middleware components in the pipeline.
2. **UseCors()**: This must come before authentication to apply the correct CORS headers.
3. **UseAuthentication()**: This must come before authorization because a user needs to be authenticated (i.e., their identity confirmed) before the system can check their permissions.
4. **UseAuthorization()**: This must come after authentication to use the newly created user identity to check against authorization policies.
5. **MapControllers()**: This is the final step, where the request is finally handed off to the appropriate controller action.

In addition, we add a custom middle before the UseAuthentication() middle for debugging purpose. It is commented out in the demo code. If you encounter bugs, you can comment it back and study the below code and its console output.

```
120  app.Use(async (context, next) =>
121  {
122      if (context.Request.Headers.TryGetValue("Authorization", out var authHeader))
123      {
124          var headerValue = authHeader.FirstOrDefault();
125          if (headerValue?.StartsWith("Bearer ") == true)
126          {
127              var token = headerValue.Substring("Bearer ".Length).Trim();
128
129              // Decode the token to see its contents (without verification)
130              var handler = new JwtSecurityTokenHandler();
131              var jsonToken = handler.ReadJwtToken(token);
132
133              Console.WriteLine($"--> Token Issuer: {jsonToken.Issuer}");
134              Console.WriteLine($"--> Token Audience: {jsonToken.Audiences.FirstOrDefault()}");
135              Console.WriteLine($"--> Token Expiry: {jsonToken.ValidTo}");
136              Console.WriteLine($"--> Current Time: {DateTime.UtcNow}");
137              Console.WriteLine($"--> Config Issuer: {builder.Configuration["Jwt:Issuer"]}");
138              Console.WriteLine($"--> Config Audience: {builder.Configuration["Jwt:Audience"]}");
139          }
140      }
141      await next.Invoke();
142  });
143  app.UseAuthentication();
144  app.UseAuthorization();
145  app.MapControllers();
```

This is a **custom middleware** that intercepts every incoming request to manually inspect the JWT token for debugging purposes.

Explanation

The app.Use() method adds a middleware to the request pipeline. This middleware:

1. **Checks for an Authorization Header:** It looks for a header named Authorization.
2. **Extracts the Token:** If the header exists and starts with Bearer , it extracts the JWT token string.
3. **Decodes and Logs Claims:** It uses a JwtSecurityTokenHandler to read and decode the token without validating its signature. It then logs key information like the token's **issuer**, **audience**, and **expiry time**. This helps you quickly verify if the token being sent from the client matches your server's expectations.

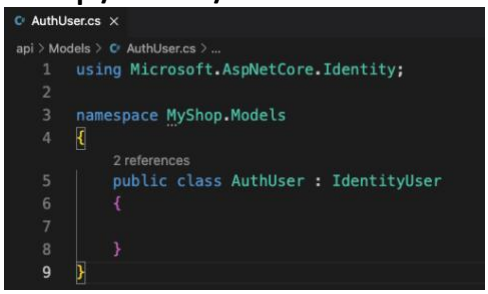
Order

The order of this custom middleware is crucial:

- It must be placed **before UseAuthentication()** and **UseAuthorization()**. This ensures that the middleware runs and logs the token information *before* the built-in authentication services validate the token. This allows you to inspect the raw token and see potential issues with its claims, such as an incorrect issuer or an expired timestamp, which could cause a validation failure later in the pipeline.

Add Model

add **api/Models/AuthUser.cs** as below



The **AuthUser.cs** class is empty because it inherits all the necessary properties for a user from the base IdentityUser class.

Why It's Empty

The **IdentityUser** class, which comes from the **Microsoft.AspNetCore.Identity** package, already contains all the standard properties a user needs, such as:

- **Id**: A unique string to identify the user.
- **UserName**: The user's login name.
- **Email**: The user's email address.
- **PhoneNumber**: The user's phone number.
- **PasswordHash**: A secure hash of the user's password.

By inheriting from **IdentityUser**, your **AuthUser** class automatically gets all these properties, so you don't have to redefine them.

You are free to add more custom properties, such as FirstName, LastName, etc.

Add DbContext

We add **api/DAL/AuthDbContext.cs**



The **AuthDbContext** is empty because it inherits all the necessary database management logic for the Identity system from the **IdentityDbContext** base class.
This is the standard and professional way to use ASP.NET Core Identity. By inheriting from **IdentityDbContext<AuthUser>,** your **AuthDbContext** automatically:

- Manages all the required tables for users, roles, and claims (like **AspNetUsers**, **AspNetRoles**, etc.).
- Connects these tables to your custom **AuthUser** model, which is a key part of your application.

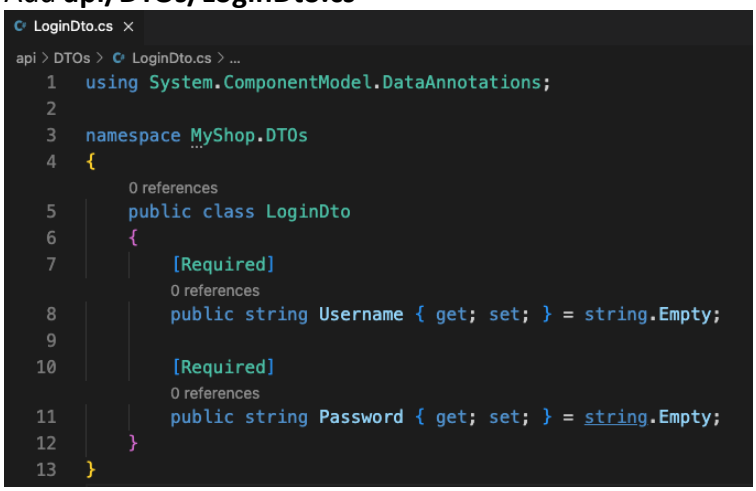
The constructor `public AuthDbContext(DbContextOptions<AuthDbContext> options) : base(options)` is also a standard pattern that allows the **DbContext** to receive its configuration (such as the connection string) through dependency injection, keeping your code clean and flexible.

After that, run the following command to add the **AuthDbContext** via migrations.

```
dotnet ef migrations add InitialAuthDb -c AuthDbContext
dotnet ef database update -c AuthDbContext
```

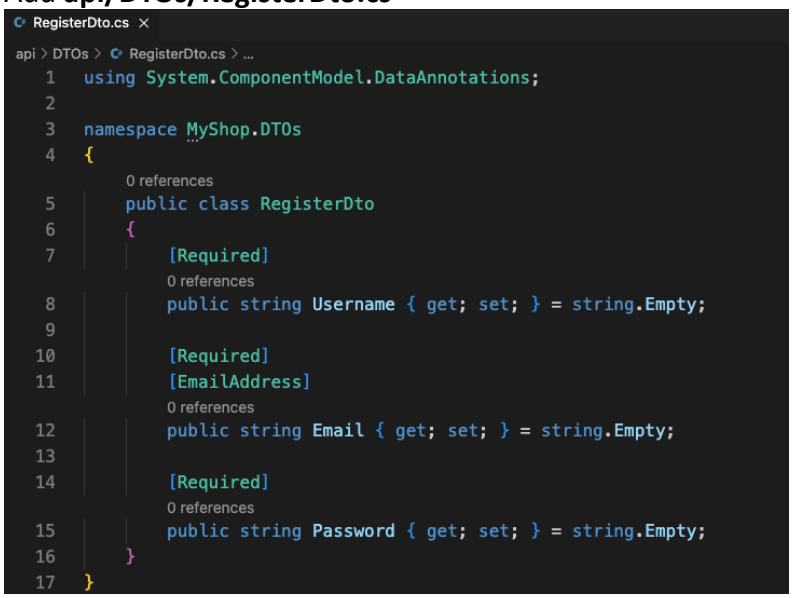
Add DTOs

Add **api/DTOs/LoginDto.cs**



The **LoginDto.cs** file defines a **Data Transfer Object (DTO)**, which is a simple class used to transfer data between the client (your React app) and the server (your API).
This specific DTO is designed to hold the username and password that a user enters on the login form. The **[Required]** attributes ensure that both the Username and Password fields must have a value when a login request is sent to the API, so the server can validate the incoming data and provide a clear error message if something is missing.

Add **api/DTOs/RegisterDto.cs**



RegisterDto.cs is a **Data Transfer Object (DTO)** used to carry data from a user's registration form to your backend API.
Its main purpose is to define the exact data fields—**Username, Email, and Password**—that a user must provide to create a new account. The attributes like **[Required]** and **[EmailAddress]** are part of a feature called **Data Annotations**. They perform a basic validation check to ensure that the fields are not empty and that the Email field is in a valid format. This simple validation happens automatically on the server before your controller code is even executed, providing a clean and efficient way to handle common validation rules.

Add AuthController

Add the api/Controllers/AuthController.cs

In the previous tutorial, we intentionally keep both `ItemAPIController.cs` and `ItemController.cs` inside `ItemController.cs` so you can directly contrast how an **API Controller (JSON & Status Codes)** differs from an **MVC Controller (Views & Actions)** without switching back and forth between tabs.
However, in professional production environments, **API endpoints and SSR controllers are kept strictly separate**. API endpoints use stateless JWT authentication and return raw JSON data, whereas MVC controllers manage session state, cookies, and HTML views.
To follow real-world architectural standards, we now build a dedicated, standalone `AuthController.cs` to manage client authentication independently of the MVC `AccountController`.

AuthController.cs

api > Controllers > AuthController.cs > AuthController > GenerateJwtToken

```
1   using Microsoft.AspNetCore.Authorization;
2   using Microsoft.AspNetCore.Identity;
3   using Microsoft.AspNetCore.Mvc;
4   using Microsoft.IdentityModel.Tokens;
5   using MyShop.DTOs;
6   using MyShop.Models;
7   using System.IdentityModel.Tokens.Jwt;
8   using System.Security.Claims;
9   using System.Text;
10
11  namespace MyShop.Controllers
12  {
13      [ApiController]
14      [Route("api/[controller]")]
        3 references
15      public class AuthController : ControllerBase // ControllerBase is sufficient for API controllers
16      {
            4 references
17          private readonly UserManager<AuthUser> _userManager;
            2 references
18          private readonly SignInManager<AuthUser> _signInManager;
            4 references
19          private readonly IConfiguration _configuration;
            8 references
20          private readonly ILogger<AuthController> _logger;
21
            0 references
22          public AuthController(UserManager<AuthUser> userManager, SignInManager<AuthUser> signInManager,
            IConfiguration configuration, ILogger<AuthController> logger)
23          {
24              _userManager = userManager;
25              _signInManager = signInManager;
26              _configuration = configuration;
27              _logger = logger;
28          }
29
30          [HttpPost("register")]
            0 references
31          public async Task<IActionResult> Register([FromBody] RegisterDto registerDto)…
52
53          [HttpPost("login")]
            0 references
54          public async Task<IActionResult> Login([FromBody] LoginDto loginDto)…
68
69          [Authorize]
70          [HttpPost("logout")]
            0 references
71          public async Task<IActionResult> Logout()…
78
            1 reference
79          private string GenerateJwtToken(AuthUser user)…
109     }
110 }
```

The **AuthController.cs** is a central hub for all user-related actions, handling registration, login, and token generation for your API. It's a standard and professional way to manage authentication in a modern application.

Key Components

- **ControllerBase**: The class inherits from ControllerBase, which is the correct choice for a backend API that sends data (like a JWT) without Views.
- **Dependency Injection**: The controller uses **dependency injection** to get instances of UserManager, SignInManager, IConfiguration, and ILogger. These services are provided by ASP.NET Core Identity to handle core user management, configuration settings, and logging.
- **[ApiController] and [Route]**: These attributes configure the controller to act as an API and define its base URL, which is **api/Auth.**
- The **AuthController** has four important endpoints. An overview is here. Details follow later.
    - **Register Endpoint**: This POST endpoint handles new user creation.
    - **Login Endpoint**: This POST endpoint handles user authentication.
    - **Logout Endpoint**: This endpoint is included for completeness, but its primary function in a token-based system is to provide a confirmation response. The actual "logout" action is performed by the client, which simply discards the token.
    - **GenerateJwtToken**: This private helper method is responsible for creating the JWT.

Dependency Injection

Using **dependency injection (DI)** to get the four dependencies is a professional and highly recommended practice in modern .NET development. It makes your code cleaner, more testable, and easier to manage. It also makes your application more secure because you're leveraging Microsoft's built-in, mature technology.

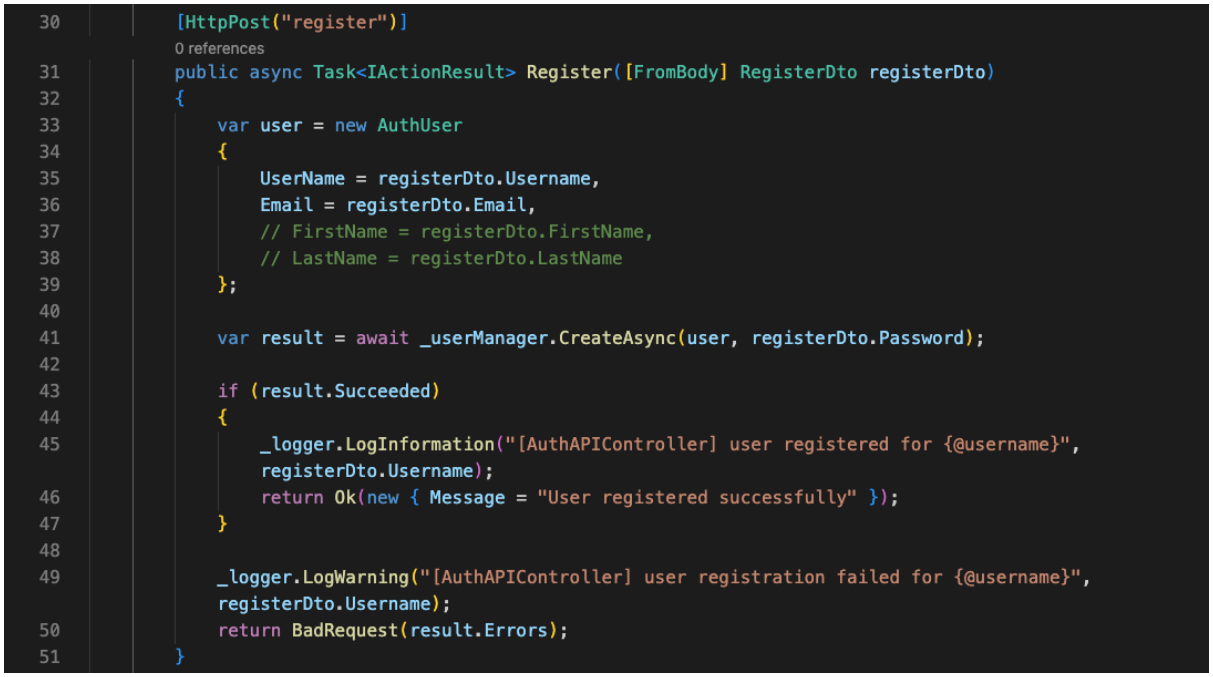
The Four Dependencies

1. **UserManager<AuthUser>**: This is the core service for user management. It securely handles critical tasks like **password hashing, salting, and validation**. By using this, you avoid writing your own code for these sensitive operations, which helps protect against common vulnerabilities like weak password storage.
2. **SignInManager<AuthUser>**: This service manages the user's sign-in flow. It's built to securely handle session management and helps implement security features like two-factor authentication.
3. **IConfiguration**: This interface provides secure access to your application's settings, like the JWT secret key. It allows you to externalise sensitive information from your source code into a secure configuration file or environment variable.
4. **ILogger<AuthController>**: This service is used for writing log messages. It's crucial for security auditing, allowing you to monitor and trace authentication events like failed login attempts, which can help detect potential attacks.

Reminder: What is Dependency Injection?

Dependency Injection is a software design pattern where an object or function receives the objects it depends on from an external source rather than creating them itself. In ASP.NET Core, the built-in DI container (the "external source") is responsible for this.

Register Endpoint

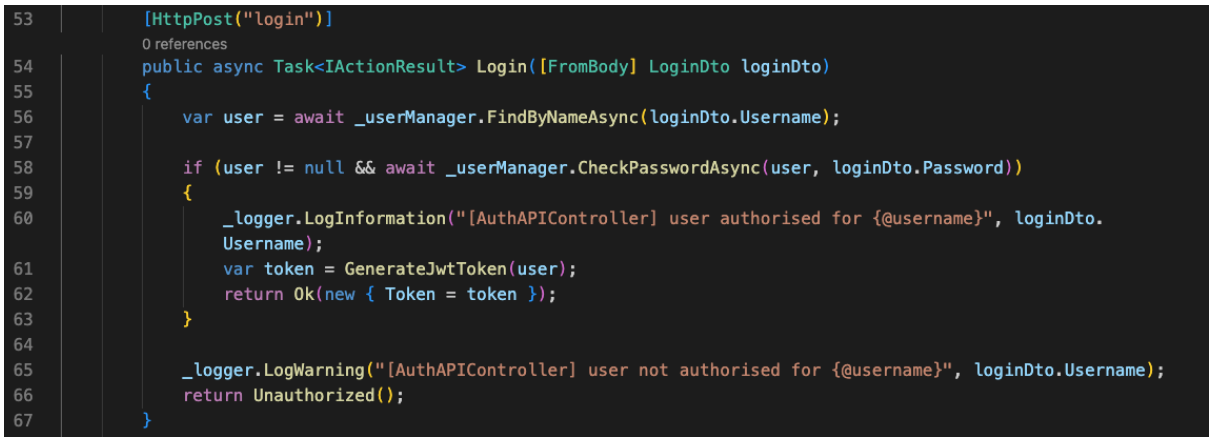


```
30      [HttpPost("register")]
        0 references
31      public async Task<IActionResult> Register([FromBody] RegisterDto registerDto)
32      {
33          var user = new AuthUser
34          {
35              UserName = registerDto.Username,
36              Email = registerDto.Email,
37              // FirstName = registerDto.FirstName,
38              // LastName = registerDto.LastName
39          };
40
41          var result = await _userManager.CreateAsync(user, registerDto.Password);
42
43          if (result.Succeeded)
44          {
45              _logger.LogInformation("[AuthAPIController] user registered for {@username}",
                registerDto.Username);
46              return Ok(new { Message = "User registered successfully" });
47          }
48
49          _logger.LogWarning("[AuthAPIController] user registration failed for {@username}",
            registerDto.Username);
50          return BadRequest(result.Errors);
51      }
```

This code defines a Register endpoint that creates a new user account.
It works by:

- Receiving user data (username, email, password) from the client via the **RegisterDto** object.
- Creating a new **AuthUser** object using the received data.
- Calling **_userManager.CreateAsync** to securely create the user in the database. This method automatically handles **password hashing** and **salting**.
- Checking if the registration was successful. If **result.Succeeded** is true, it returns a **200 OK** status with a success message.
- If the registration fails (e.g., the username is already taken or the password is too weak), it logs a warning and returns a **400 Bad Request** with a list of the specific errors.

Login Endpoint

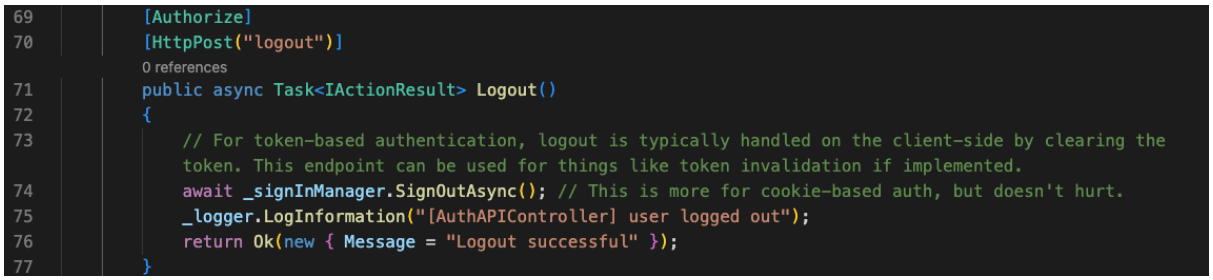


```
53      [HttpPost("login")]
        0 references
54      public async Task<IActionResult> Login([FromBody] LoginDto loginDto)
55      {
56          var user = await _userManager.FindByNameAsync(loginDto.Username);
57
58          if (user != null && await _userManager.CheckPasswordAsync(user, loginDto.Password))
59          {
60              _logger.LogInformation("[AuthAPIController] user authorised for {@username}", loginDto.
                Username);
61              var token = GenerateJwtToken(user);
62              return Ok(new { Token = token });
63          }
64
65          _logger.LogWarning("[AuthAPIController] user not authorised for {@username}", loginDto.Username);
66          return Unauthorized();
67      }
```

This code defines the **Login endpoint**, which handles user authentication and, if successful, issues a JWT. Here's how it works:

- It receives the login credentials from the client via a **LoginDto** object.
- It uses **_userManager.FindByNameAsync** to find the user in the database.
- It then uses **_userManager.CheckPasswordAsync** to securely validate the provided password against the stored hashed password.
- If the username and password are correct, it logs a success message, calls **GenerateJwtToken** to create a JWT, and returns the token to the client with a **200 OK** status.
- If the credentials are not correct, it logs a warning and returns a **401 Unauthorized** status. This generic error message is a security best practice as it prevents attackers from knowing whether a username exists or not.

Logout Endpoint



```
69      [Authorize]
70      [HttpPost("logout")]
        0 references
71      public async Task<IActionResult> Logout()
72      {
73          // For token-based authentication, logout is typically handled on the client-side by clearing the
            token. This endpoint can be used for things like token invalidation if implemented.
74          await _signInManager.SignOutAsync(); // This is more for cookie-based auth, but doesn't hurt.
75          _logger.LogInformation("[AuthAPIController] user logged out");
76          return Ok(new { Message = "Logout successful" });
77      }
```

This endpoint handles the logout process for a user. While it's included for completeness, its primary purpose in a JWT-based API is to provide a confirmation response, as the actual "logout" is performed on the client.

- **[Authorize]**: This attribute ensures that only authenticated users can access this endpoint. If a request does not have a valid JWT token, it will be rejected automatically with a **401 Unauthorized** status.
- **_signInManager.SignOutAsync()**: This method is part of ASP.NET Core Identity's cookie-based authentication system. It clears the authentication cookie from the server, ending the user's session. In a pure JWT scenario, **this action doesn't affect the token (which is stateless), but it doesn't cause any harm**.

- **Return Response**: The method returns a **200 OK** status with a simple message, confirming to the client that the logout action was handled. In a JWT-based system, the client is responsible for discarding the token to effectively log out.

GenerateJwtToken Endpoint



```
1 reference
private string GenerateJwtToken(AuthUser user)
{
    var jwtKey = _configuration["Jwt:Key"]; // The secret key used for the signature
    if (string.IsNullOrEmpty(jwtKey)) // Ensure the key is not null or empty
    {
        _logger.LogError("[AuthAPIController] JWT key is missing from configuration.");
        throw new InvalidOperationException("JWT key is missing from configuration.");
    }
    var securityKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)); // Reading the key from the
    configuration
    var credentials = new SigningCredentials(securityKey, SecurityAlgorithms.HmacSha256); // Using HMAC
    SHA256 algorithm for signing the token

    var claims = new[]
    {
        new Claim(JwtRegisteredClaimNames.Sub, user.UserName!), // Subject of the token
        new Claim(JwtRegisteredClaimNames.Email, user.Email!), // User's email
        new Claim(ClaimTypes.NameIdentifier, user.Id), // Unique identifier for the user
        new Claim(JwtRegisteredClaimNames.Jti, Guid.NewGuid().ToString()), // Unique identifier for the token
        new Claim(JwtRegisteredClaimNames.Iat, DateTimeOffset.UtcNow.ToUnixTimeSeconds().ToString()) //
        Issued at timestamp
    };

    var token = new JwtSecurityToken(
        issuer: _configuration["Jwt:Issuer"],
        audience: _configuration["Jwt:Audience"],
        claims: claims,
        expires: DateTime.Now.AddMinutes(120), // Token expiration time set to 120 minutes
        signingCredentials: credentials); // Signing the token with the specified credentials

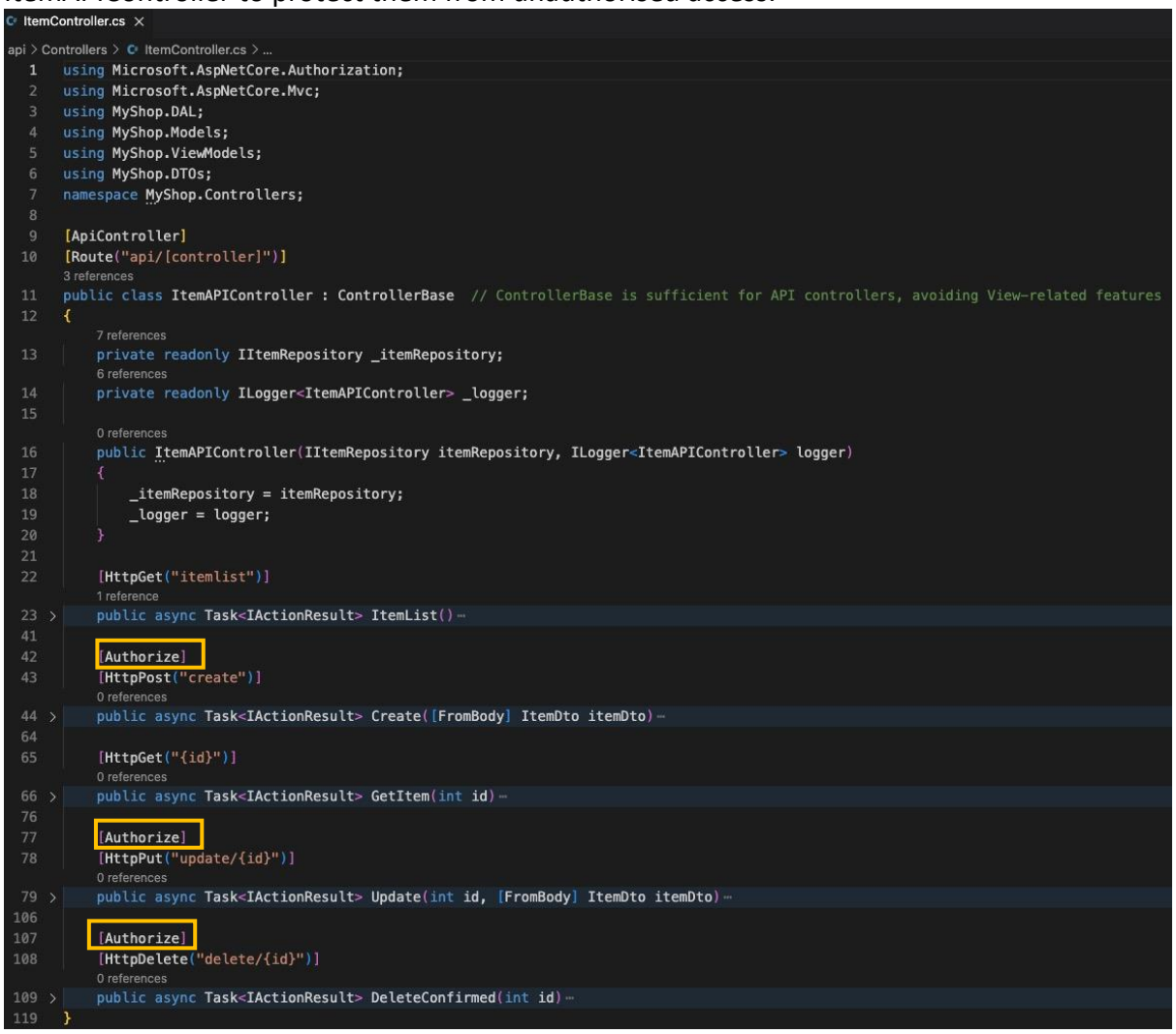
    _logger.LogInformation("[AuthAPIController] JWT token created for {@username}", user.UserName);
    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

This method, **GenerateJwtToken**, is responsible for creating a **JSON Web Token (JWT)** for a successfully authenticated user.

- **Security Setup**: It first retrieves the secret key from your **appsettings.json** file. It then uses this key to create a **SymmetricSecurityKey** and **SigningCredentials**, which are essential for digitally signing the token. This signature ensures the token's integrity and authenticity.
- **Claims Creation**: It creates an array of **claims**, which are pieces of information about the user. These include the username, email, and unique user ID. These claims are later used by your API to identify the user and check their permissions.
- **Token Creation**: It constructs a new **JwtSecurityToken** with all the necessary components: the claims, the issuer and audience from your configuration, the expiration time, and the signing credentials.
- **Serialisation**: Finally, it uses a **JwtSecurityTokenHandler** to serialise the token object into a compact, URL-safe string that can be sent to the client. The method also includes robust error handling and logging for key events.

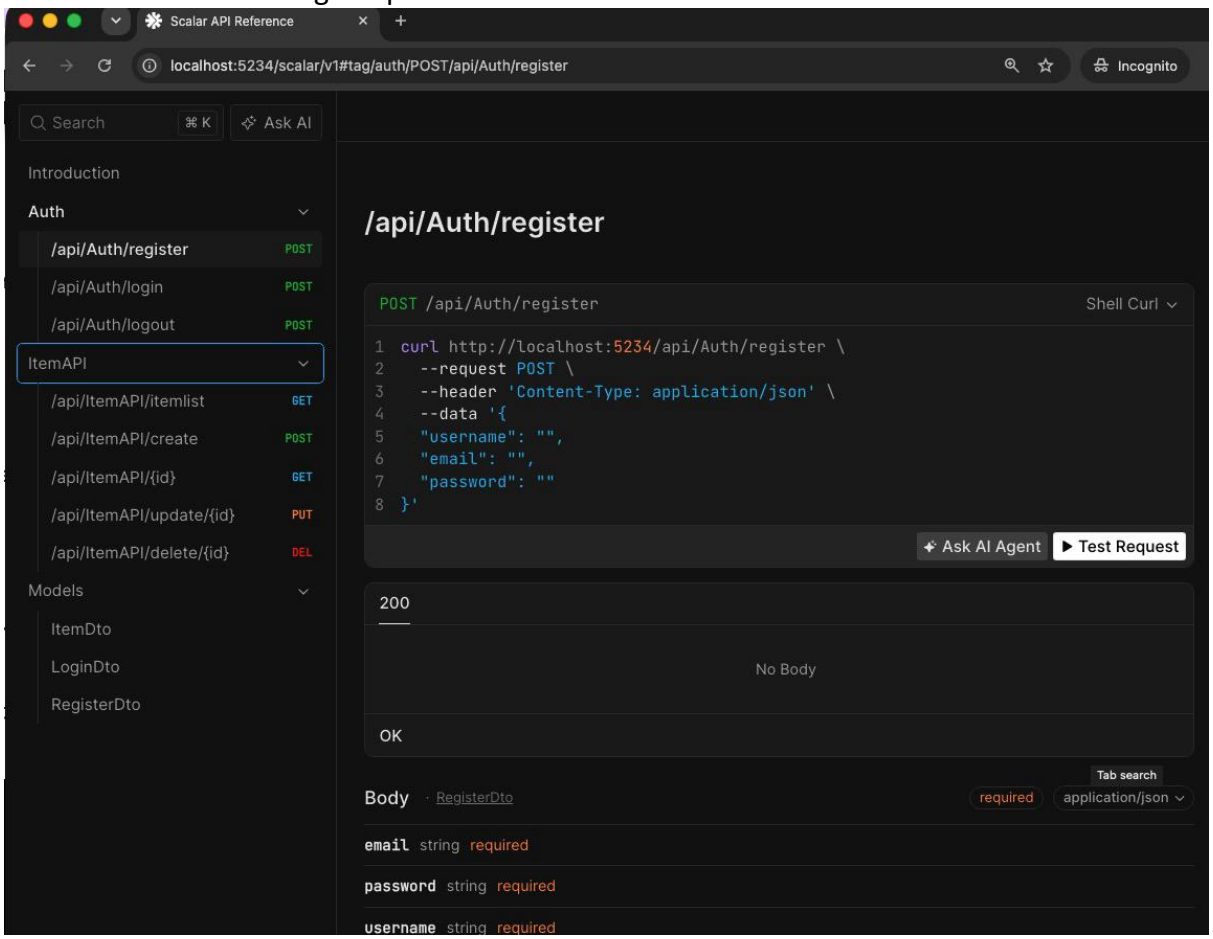
Modifying ItemController

Finally, we need to add the [Authorize] attributes to the create/update/delete actions of the ItemAPIController to protect them from unauthorised access.

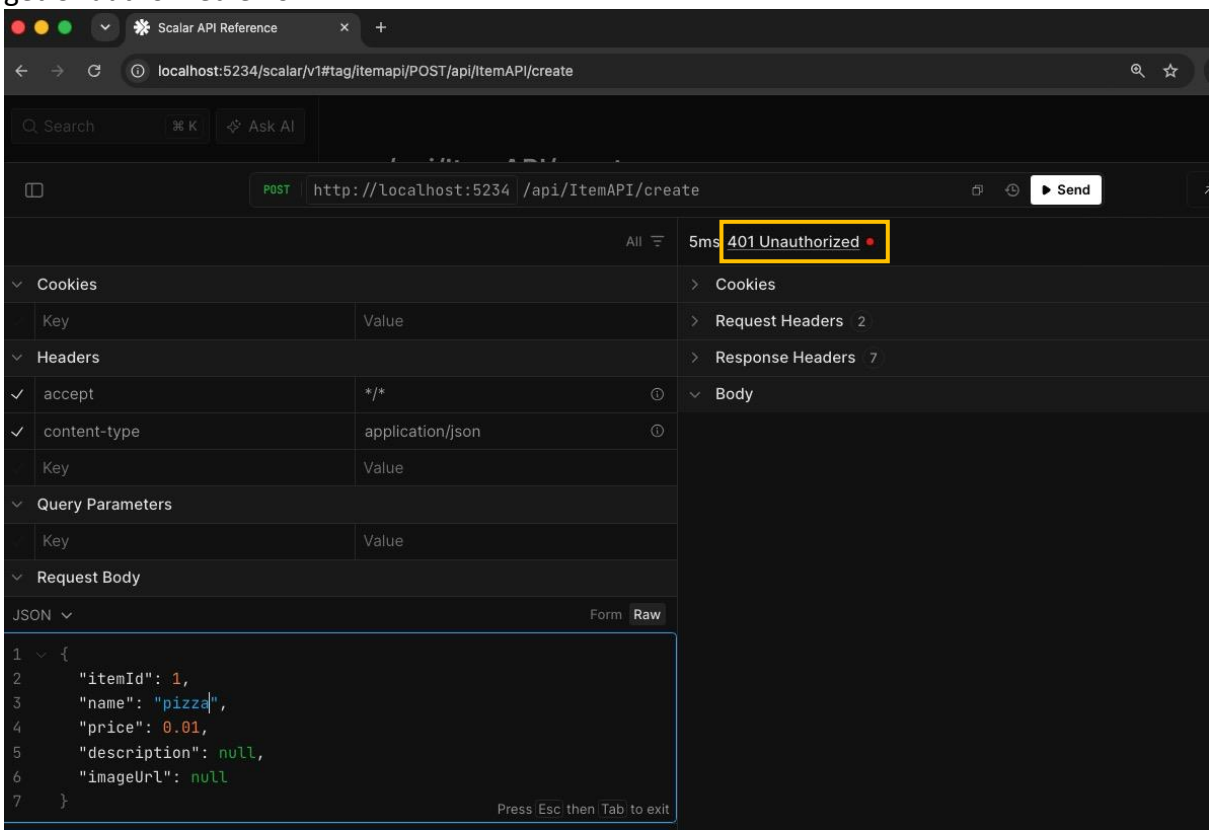


Test the Backend API via Scalar

Now we can test the **backend api** with authentication functionality with Scalar.
Start the app with **dotnet watch** under the folder **api**.
We can see the following endpoints and Models

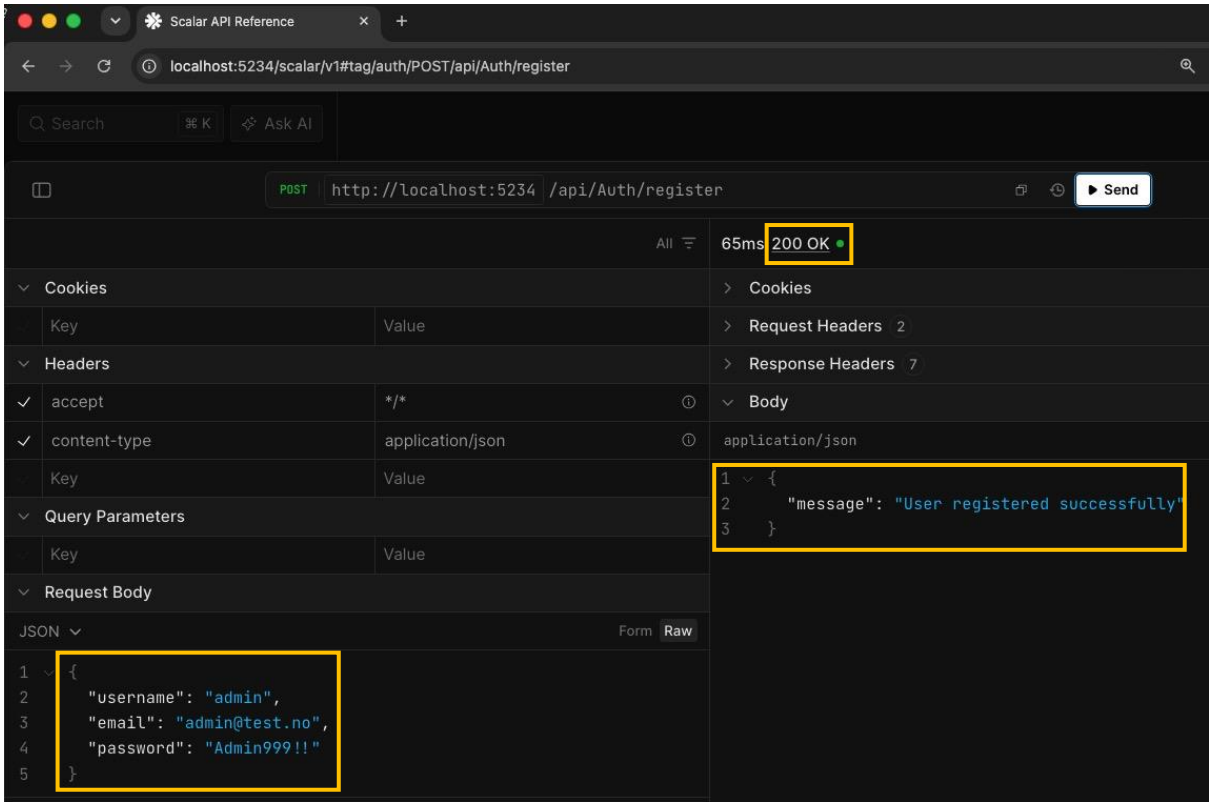


Now if you try the create/update/delete directly without logging in, and click send, you will get Unauthorized error.

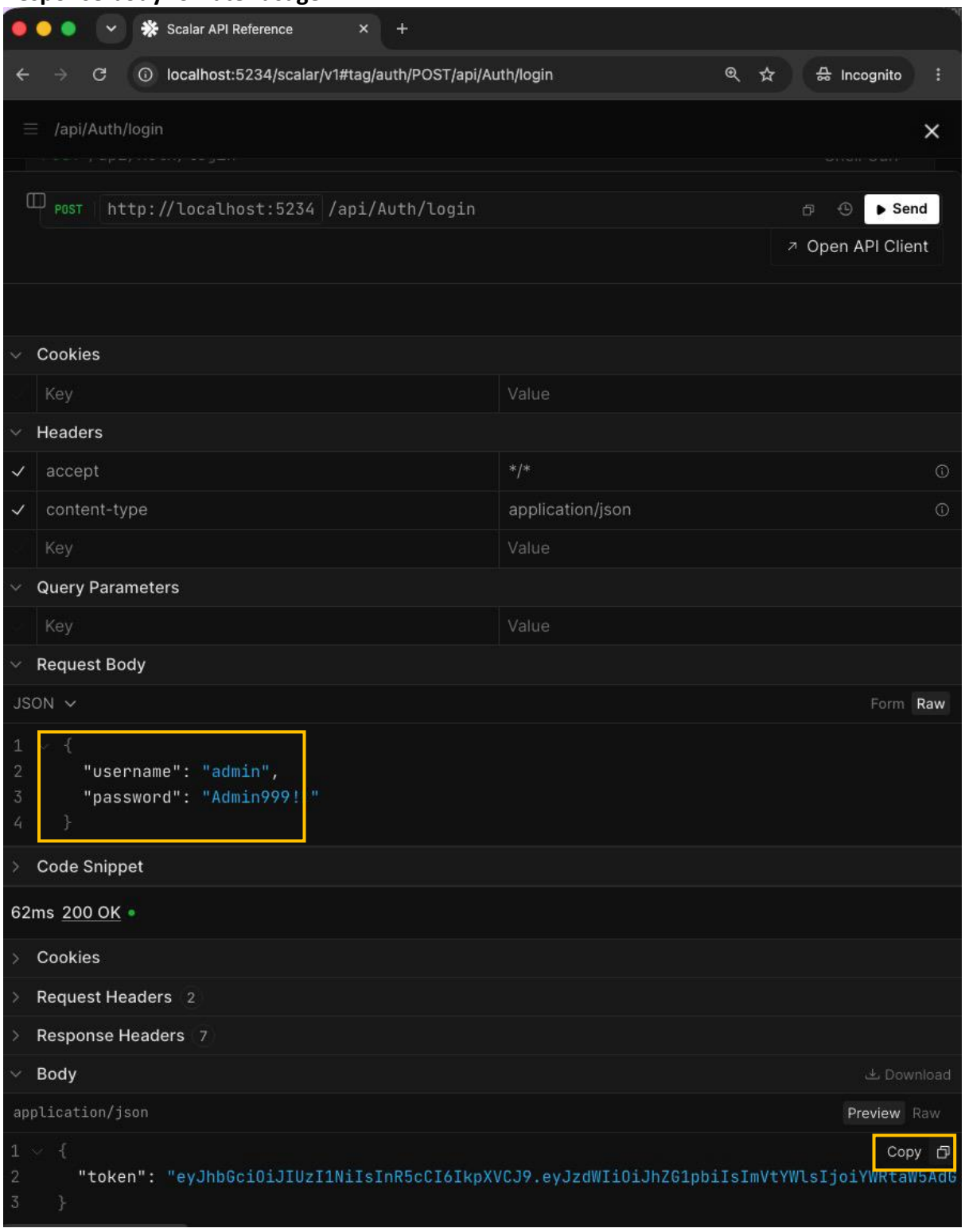


We now register a user using the /api/Auth/register endpiont with the below credentials, then we can see the return code and message body.

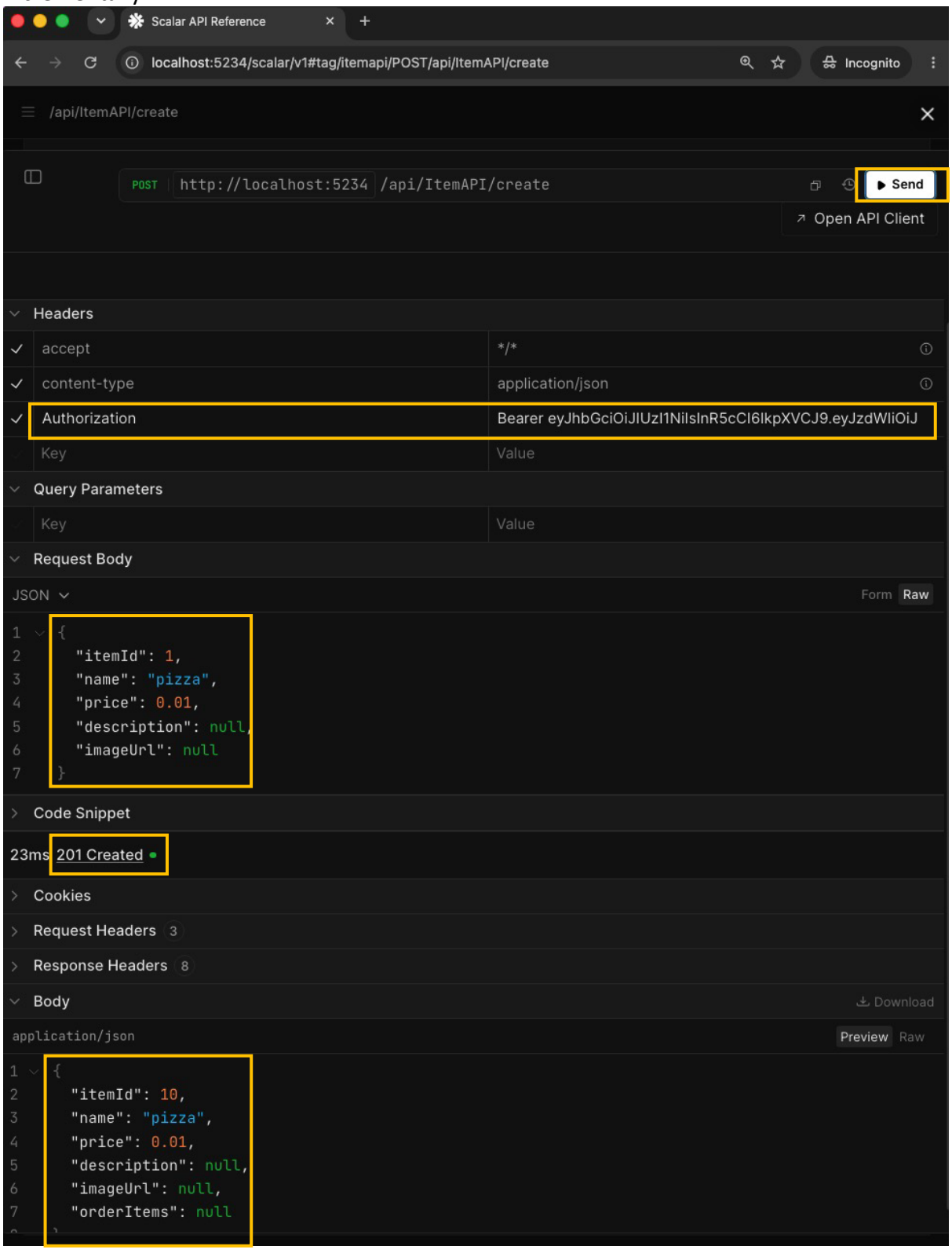
```
{
  "username": "admin",
  "email": "admin@test.no",
  "password": "Admin999!!"
}
```



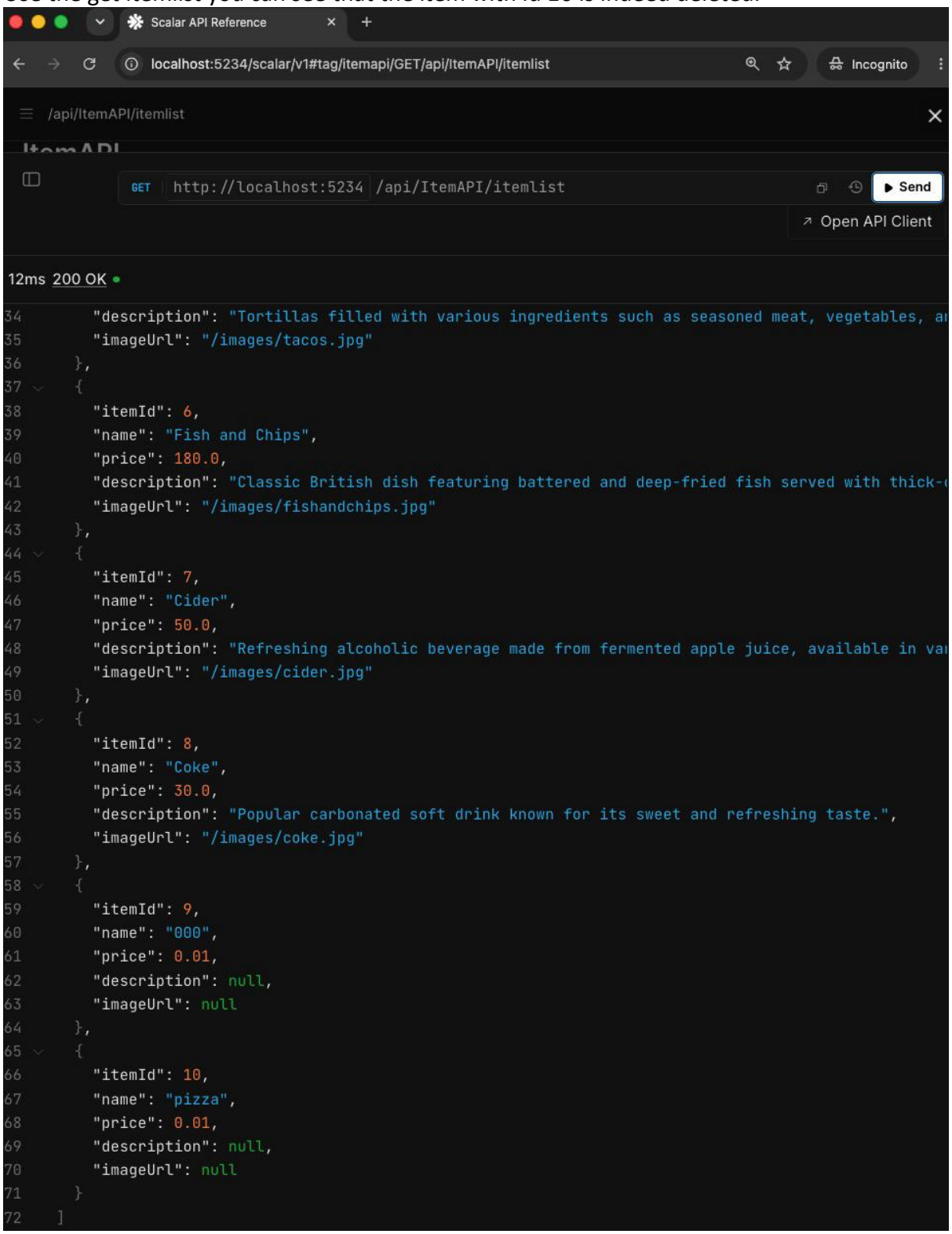
And login using the /api/Auth/login endpoint. **We need to copy the JWT token from the Response body for later usage**.



In api/ItemAPI/create ,add a new key in Headers, and in the value cell after it, enter:
"`Bearer <JWT token copied from the login Response body>`"
Note there is a space between the "Bearer" and the token.
Also add a valid data entry in the Request Body, Click send, you can see the it is successful. The itemId is automatically changed to 10 because the database will assign the id incrementally.



Use the get itemlist you can see that the item with Id 10 is indeed deleted.



Congratulations, you have accomplished the Backend API authentication! ☺ Really great achievement!